

CSF2753 Lab Class

Filippos Pantekis

CSF2753 Lab Class 2

Wednesday 30/09/2026

In this lab we will exploring the application of Threads in Java and some of the different ways you can manage them.

I recommend creating a directory / folder for the module labs. There are some resources to get you started with solutions to Task 1. They can be found under:

Modules → Week 2 → Lab

☐ Task 2.1

First we're going to use a class extending `Thread` to sum up a sequence of squared numbers. The starter code for this can be found in `SumOfSquaresThread.java` in the `Lab 2.zip` file on Canvas.

The `SumOfSquaresThread` class is quite simple. It receives two integers which define a range of numbers to square and sum up. It implements this in the `run()` method. It also has a `getResult()` method for returning the sum once the thread is done.

Write a main method that creates and runs at least **two** threads to perform this task. You can use your own (reasonably) arbitrary ranges for the constructor. Once the threads have finished, print out the sum of their results. Explain the result. Is it consistent? Is this safe from race conditions?

☐ Task 2.2

Next, we'll use a similar example, but one that defines the task as a class implementing the `Runnable` interface. You can use the starter code in `SumOfSquaresRunnable.java` to begin with.

Write a main method that passes a single instance of the `SumOfSquaresRunnable` task to **at least** two threads. Explain the result. Does this behave differently to the code from Task 2.1? If not, why?

☐ Task 2.3

In the lecture we introduced the `synchronized` keyword in Java. This helps ensure mutual exclusion (i.e controls threads so only one thread can access a block of code at a time) across sections of our programs. Now, we'll consider how this can be applied to our `SumOfSquaresRunnable` example.

Synchronize the code from the previous task so that it returns a consistent result, regardless of the number of threads we spawn.

Please turn over ➡

□ Task 2.4

As we saw in Part 1 of today's lecture, multithreading can help make our web services more responsive. We can have a main thread listen for client requests and spawn new threads to process each connection's task.

Extend the random number generator UDP server from **Lab 1 - Task 1.1** to start a new thread once a client request is accepted. To do this, we will need a new class that extends **Thread**.

You will need at least three classes to complete the client-server interaction. Carefully consider what you want the main thread to manage and what tasks you offload to each thread.

□ Task 2.5

Consider the concepts we addressed in the lectures today. Does this implementation run the risk of race conditions? If so, why? What other problems might occur here?

□ Challenge Task 2.6

Once you have implemented your multithreaded server, consider whether this is actually improving the client-server interaction (as we currently have it set up). Is it doing anything useful? Well, not really.

The point of having the ability of spinning up new threads on our server is to handle multiple concurrent requests, and with the way our client works, this isn't happening. So, let's change that.

Extend your client code to send threaded requests to your newly threaded server. Explain what is happening. How does this simulate "real" distributed programming?